Senior Developer Deep-Dive & Tricky Interview Questions

This document contains advanced interview questions commonly asked to 5–6+ years experienced developers. These questions test in-depth understanding of programming languages, databases, concurrency, distributed systems, JVM internals, APIs, and BFSI-related engineering concepts.

Java / Programming Internals

1. Why does String.length() return 2 for certain Unicode characters?
2. Why is String immutable?
3. Explain Integer caching in Java.
4. How does HashMap internally work?
5. Explain equals() vs hashCode() contract.
6. Why can HashMap fail in multithreaded environments?
7. What happens when return statements exist in both try and finally?
8. What happens if a mutable object is used as HashMap key?
9. Difference between shallow copy and deep copy.
10. Difference between == and equals().
11. What is string pool and why does JVM maintain it?
12. Why are enums considered thread-safe?
13. Difference between checked and unchecked exceptions.
14. Why is Optional controversial in Java?
15. What is autoboxing and why can it cause performance issues?

JVM / Memory Management

1. Explain stack vs heap memory.
2. Can Java have memory leaks? Give real examples.
3. Why was finalize() deprecated?
4. How does garbage collection work?
5. Difference between Minor GC and Major GC.
6. What causes OutOfMemoryError?
7. What causes StackOverflowError?
8. How would you analyze heap dump?

9. What are soft, weak, and phantom references?

10. What is Metaspace?

Multithreading & Concurrency

1. Difference between volatile and synchronized.

2. Explain happens-before relationship.

3. Why was double-checked locking broken before volatile fix?

4. How does deadlock occur?

5. Difference between starvation and deadlock.

6. What is race condition?

7. Difference between Callable and Runnable.

8. How does ConcurrentHashMap work internally?

9. What is thread-local storage?

10. Why is immutability useful in concurrent systems?

Database & SQL

1. Difference between WHERE and HAVING.

2. DELETE vs TRUNCATE vs DROP.

3. Why does query ignore indexes sometimes?

4. Explain ACID properties with examples.

5. Difference between dirty read, phantom read, and non-repeatable read.

6. What is normalization and denormalization?

7. How do indexes improve performance?

8. What causes database deadlocks?

9. What is optimistic vs pessimistic locking?

10. How would you optimize a slow SQL query?

APIs & Distributed Systems

1. Why should payment APIs be idempotent?

2. Difference between PUT and PATCH.

3. Why are distributed transactions difficult?

4. Explain eventual consistency.

5. What is circuit breaker pattern?

6. What causes duplicate payment transactions?

7. How does retry logic create issues?

8. Difference between synchronous and asynchronous communication.

9. What is message ordering problem in Kafka?

10. What happens during network partition?

Microservices & System Design

1. When should microservices NOT be used?

2. How do you handle distributed logging?

3. What is service discovery?

4. How does API Gateway work?

5. Why can microservices increase complexity?

6. How would you design a scalable payment system?

7. How do you avoid cascading failures?

8. What are eventual consistency tradeoffs?

9. What is backpressure?

10. How would you debug latency spikes?

Production Debugging & Real Engineering

1. Why does issue reproduce only in production?

2. How would you debug sudden CPU spike?

3. Why system works for 1K users but fails for 100K users?

4. How do connection pool leaks happen?

5. How would you analyze GC pauses?

6. How do memory leaks happen in enterprise apps?

7. What metrics are critical in production monitoring?

8. How do you identify bottlenecks?

9. How would you debug intermittent failures?

10. What was the toughest production bug you fixed?

BFSI / Banking Domain Questions

1. Why is idempotency critical in banking systems?
2. Why are duplicate payment prevention systems difficult?
3. How does reconciliation work in banking?
4. What are AML and KYC systems?
5. Why are audit trails critical in BFSI?
6. Why do banking systems require strong consistency?
7. How do settlement systems work?
8. Why are timezones important in financial systems?
9. How do banks ensure transactional integrity?
10. What are common fraud detection patterns?

These questions are valuable because they test engineering maturity, debugging capability, real-world production experience, and deep conceptual understanding — not just memorized syntax.